SICLAW: Defense-in-Depth Security Architecture for Production LLM-Based SRE Agents

Anonymous submission

Abstract

Large language model (LLM) agents are increasingly applied to Site Reliability Engineering (SRE) tasks such as incident diagnosis, root cause analysis, and remediation in Kubernetes environments. However, existing SRE agent systems—including Stratus, FLASH, and RCAgent—treat the LLM as trusted code, exposing production infrastructure to prompt injection, credential exfiltration, and privilege escalation attacks. We present SICLAW, the first production-grade SRE agent system that treats the LLM as *untrusted code* and enforces safety through a six-layer defense-in-depth architecture. SICLAW introduces: (1) an OS-level dual-user isolation model with a novel setgid-based transitive privilege mechanism for selective credential access; (2) a six-pass command validation pipeline that decouples command definitions from execution-context policies; (3) a three-layer output sanitization pipeline preventing credential leakage through tool outputs; (4) a four-stage guard pipeline addressing LLM output unreliability (malformed tool calls, context overflow, stream corruption); and (5) an extensible skill architecture with three-gate review for organizational knowledge accumulation. SICLAW supports three runtime modes—single-user TUI, multi-user local development, and production Kubernetes with per-user pod isolation—sharing a unified agent core. Evaluated on 100 Kubernetes fault diagnosis cases spanning 10 categories (including GPU scheduling and multi-fault compound scenarios), SICLAW achieves a 90% pass rate with an average checklist score of 0.818 across five diagnostic dimensions. In a 30-scenario red-team evaluation, the security architecture blocks 100% of attacks across four threat categories, with a layer ablation confirming that no single layer exceeds 70% coverage—validating the defense-in-depth design. Critically, a tradeoff analysis shows the security constraints impose *zero* accuracy cost: none of the 10 failed cases were caused by security restrictions, while an unconstrained agent would generate an estimated 2.4 security violations per case. Additionally, we present the first evaluation of an SRE agent on GPU hardware and RDMA network fault scenarios — including Xid ECC errors, NVLink degradation, PFC storms, and compound GPU+RDMA failures — achieving 100% pass rate (10/10) with an average score of 0.923. To our knowledge, SICLAW is the first SRE agent system to formalize a threat model for LLM infrastructure agents and implement a comprehensive defense-in-depth security architecture.

Introduction

The rapid advancement of large language models (LLMs) has sparked significant interest in applying LLM-based agents to Site Reliability Engineering (SRE) tasks (Chen et al. 2025b,a; Zhang et al. 2024). These agents promise to automate incident diagnosis, root cause analysis (RCA), and remediation in complex cloud-native environments, potentially reducing mean time to resolution (MTTR) and alleviating the chronic shortage of experienced SRE practitioners.

Recent systems have demonstrated encouraging results. Stratus (Chen et al. 2025a) introduces multi-agent collaboration with undo-and-retry mechanisms, achieving 78.5% mitigation success on the SREGym benchmark (Clark et al. 2026). FLASH (Zhang et al. 2024) decomposes diagnostic workflows into conditional segments with hindsight generation, reaching 59.3% accuracy on AIOpsLab (Chen et al. 2025b). Evaluation frameworks such as AIOpsLab (Chen et al. 2025b), SREGym (Clark et al. 2026), and IT-Bench (Jha et al. 2025) have established rigorous benchmarks for measuring agent capabilities.

However, a critical gap exists between these research prototypes and production-deployable systems. **No existing SRE agent system treats the LLM as untrusted code.** AIOpsLab mentions "security policy filters" for shell execution without specification; SREGym exposes unrestricted `kubectl` access ("cluster control for executing any commands"); Stratus, FLASH, and RCAgent (Wang et al. 2024) provide no documented security model. This is deeply problematic: LLM agents are susceptible to prompt injection (Ruan et al. 2024), hallucination-driven command generation, and adversarial manipulation—attacks that in an infrastructure context can lead to credential theft, data exfiltration, or cluster-wide outages.

We identify five critical gaps in the current SRE agent literature (Table 1):

1. **Agent security**: No system provides defense-in-depth against the agent's own generated commands.
2. **Multi-tenant isolation**: All systems assume single-user, single-agent scenarios.
3. **Output sanitization**: No system prevents sensitive data (credentials, tokens) in command outputs from reaching the LLM's context.

Table 1: Capability comparison of SRE agent systems. SICLAW is the only system addressing agent security, multi-tenant isolation, output sanitization, and extensible knowledge management.

Capability	*AIOpsLab*	*SREGym*	*Stratus*	*FLASH*	*SICLAW*
Live K8s environment	✓	✓	✓	✓†	✓
Agent security model	—	—	—	—†	**6-layer**
Multi-tenant isolation	—	—	—	—	**3 modes**
Output sanitization	—	—	—	—	**3-layer**
Extensible skills	—	—	—	fixed	**4-tier**
Guard pipeline	—	—	—	—	**4-stage**
GPU/RDMA fault eval.	—	—	—	—	**10 cases**

†Proprietary; not reproducible.

4. **Agent reliability**: LLM output unreliability (malformed tool calls, context overflow) is documented by every benchmark but addressed by none.
5. **Knowledge extensibility**: Existing systems use fixed tool sets or static Standard Operating Procedures (SOPs), preventing organizational knowledge accumulation.

We present SICLAW, a production-grade SRE agent system that addresses all five gaps. Our key contributions are:

- **Defense-in-depth security architecture** (§): Six independent security layers, including a novel dual-user OS isolation model with setgid-based transitive privilege for selective credential access, a six-pass command validation pipeline, and a three-layer output sanitization pipeline. Each layer provides protection independently—compromising one does not disable the others.
- **Guard pipeline for agent reliability** (§): A four-stage message interception architecture (input, context, output, persist) that systematically addresses LLM output unreliability, including malformed tool call repair, context budget enforcement, and stream event correction.
- **Multi-tenant runtime polymorphism** (§): Three runtime modes sharing a unified agent core, with formal isolation invariants for shared-filesystem and per-pod deployment models.
- **Extensible skill architecture** (§): A four-tier skill hierarchy with three-gate review (static analysis, AI semantic review, human approval) enabling safe organizational knowledge accumulation.
- **Comprehensive evaluation** (§): 100 Kubernetes fault diagnosis cases across 10 categories, plus 10 GPU/RDMA infrastructure fault scenarios grounded in production data — the first such evaluation for SRE agents — with security red-team testing and tradeoff analysis.

Related Work

LLM-Based SRE Agents. The application of LLMs to operations tasks has progressed rapidly. RCACopilot (Chen et al. 2024) demonstrated LLM-based root cause analysis on Microsoft cloud incidents using in-context learning with historical reports. Xpert (Jiang et al. 2024) applied LLMs to query recommendation for incident management. RCAgent (Wang et al. 2024) introduced tool-augmented LLMs for cloud RCA with ReAct-style reasoning (Yao et al. 2023). D-Bot (Zhou et al. 2024) applied tree-of-thought reasoning to database diagnosis. FLASH (Zhang et al. 2024) introduced workflow decomposition with hindsight generation for recurring incidents. Stratus (Chen et al. 2025a) proposed multi-agent SRE with observation, diagnosis, and mitigation sub-agents, achieving the highest mitigation rate on SREGym. Flow-of-Action (Pei et al. 2025) encoded domain knowledge as SOP-guided multi-agent workflows. MonitorAssistant (Yu et al. 2024b) simplified cloud monitoring setup using LLMs. TAMO (Zhang et al. 2025) integrated multi-modal observability tools for fine-grained RCA. None of these systems address agent security, multi-tenant isolation, or output sanitization.

AIOps Benchmarks. AIOpsLab (Chen et al. 2025b) established the first holistic evaluation framework with 48 problems across DeathStarBench (Gan et al. 2019) microservices, introducing the Agent-Cloud Interface (ACI) concept. SREGym (Clark et al. 2026) extended this with 90 problems featuring noise injection, full-stack faults (including eBPF-simulated hardware failures), and an LLM-as-judge checklist oracle achieving $\kappa = 0.90$ human agreement. ITBench (Jha et al. 2025) broadened scope to general IT automation. OpenRCA (Xu et al. 2025) focused specifically on root cause localization. Cloud-OpsBench (Wang et al. 2026) targeted reproducible agentic RCA. These benchmarks evaluate diagnostic capability but do not assess agent safety properties, multi-tenant scenarios, or GPU/RDMA infrastructure faults — despite GPU hardware errors being the dominant failure mode in production AI clusters (Cui et al. 2025; Wan et al. 2025).

Agent Safety. General agent safety research has produced benchmarks such as AgentBench (Liu et al. 2024) and ToolEmu (Ruan et al. 2024), which evaluate LLM agents in sandboxed or emulated environments. However, these focus on general-purpose agent safety (harmful content generation, unauthorized actions) rather than infrastructure-specific threats (credential exfiltration, privilege escalation via `kubectl`, sensitive output leakage). To our knowledge, no prior work formalizes a threat model for LLM agents operating on production infrastructure.

Threat Model and Problem Formulation

Core Assumption. We model the LLM agent as *untrusted code* that generates shell commands executing in a container with Kubernetes cluster access. The agent may attempt—through prompt injection, hallucination, or adversarial manipulation—to: (T1) *read credentials* (kubeconfig, mTLS certificates, API keys); (T2) *exfiltrate data* to external endpoints; (T3) *escalate privileges* beyond read-only Kubernetes operations; (T4) *leak sensitive information* through command outputs that enter the LLM context.

Trust Boundaries. The AgentBox container contains two trust domains: the *main process* (Node.js runtime, LLM API client, tool validation logic) running as user `agentbox`, and *child processes* (shell commands generated by the LLM) running as user `sandbox`. The trust boundary lies at the command execution interface: the main process validates and sanitizes; child processes execute with minimal privilege.

Security Goals. We enforce three security properties:

1. **Credential isolation**: Child processes cannot read credential files, even if the command validation layer is bypassed.
2. **Least privilege**: Only explicitly whitelisted commands with approved flags can execute; `kubectl` is restricted to 13 read-only subcommands.
3. **Output safety**: Sensitive data (tokens, keys, connection strings) in command outputs is redacted before entering the LLM context.

Problem Formulation. Let an SRE agent $\mathcal{A}$ interact with a Kubernetes cluster $\mathcal{K}$ through a tool interface $\mathcal{T}$. At each step t, the agent generates a command $c_t = \mathcal{A}(h_t, o_t)$ conditioned on history h_t and current observation o_t. The command passes through a security pipeline $\mathcal{S}$: $c'_t = \mathcal{S}(c_t)$, which either approves, modifies, or blocks the command. The execution produces output $r_t = \text{exec}(c'_t)$, which passes through an output sanitizer $\mathcal{O}$: $r'_t = \mathcal{O}(r_t, c'_t)$ before being appended to the agent's context. The agent additionally passes through a guard pipeline $\mathcal{G}$ that repairs malformed messages and enforces context budgets.

Our goal is to maximize diagnostic accuracy $\text{Acc}(\mathcal{A})$ subject to the security constraints: $\forall t : \mathcal{S}(c_t)$ enforces credential isolation, least privilege, and $\mathcal{O}(r_t)$ enforces output safety.

System Architecture

SICLAW is an LLM-based SRE agent system built on a single brain abstraction (OpenAI-compatible provider interface) with a unified tool registry, guard pipeline, and security architecture. Figure 1 shows the overall system: user intent from multiple channels flows through a control plane that orchestrates per-user *AgentBox* sessions, each of which investigates read-only infrastructure targets. This section details the agent core and its security architecture (Figure 2).

Defense-in-Depth Security Architecture

The security architecture consists of six independent layers, each providing protection even if others are compromised. This section details the three most novel layers.

Layer 1: OS-Level Dual-User Isolation The AgentBox container contains two Linux users: `agentbox` (UID 1000, groups: `agentbox`, `kubecred`) runs the main Node.js process, and `sandbox` (UID 1001, group: `sandbox`) runs all LLM-generated shell commands via `sudo -E -u sandbox`.

Table 2: File permission matrix. The sandbox user cannot access any credential or configuration file. kubectl gains kubeconfig access via setgid kubecred group.

Asset	agentbox	sandbox	kubectl
Kubeconfig (0640)	rw	–	r (setgid)
mTLS certs (0600)	rw	–	–
API keys (0600)	rw	–	–
Skills (0644)	rw	r	r
User data (0777)	rwx	rwx	rwx

Transitive Privilege via setgid. A key challenge is that `kubectl` must read kubeconfig files to interact with the cluster, but the `sandbox` user must not have direct access to these credentials. We solve this with a setgid mechanism: `kubectl` is installed with the setgid bit for the `kubecred` group (`-rwxr-sr-x root:kubecred`). Kubeconfig files are owned by `agentbox:kubecred` with mode `0640`. When `sandbox` executes `kubectl`, the setgid bit temporarily grants `kubecred` group membership, allowing `kubectl` to read the kubeconfig. Other commands (`grep`, `cat`, `jq`) running as `sandbox` lack this group membership and cannot read credential files.

This design provides a key property: *pipeline compatibility*. Natural shell pipelines like `kubectl get pods | grep Error` work correctly because data flows through stdout (which is not access-controlled), while credential files remain protected by filesystem permissions.

File Permission Matrix. Table 2 shows the access control matrix. The design ensures that even if the command validation layer (Layer 2) is completely bypassed, the `sandbox` user's filesystem permissions independently prevent credential access.

Layer 2: Six-Pass Command Validation Pipeline Every LLM-generated command passes through six independent validation passes before execution (Algorithm 1):

Context-Policy Decoupling. A key design innovation is the separation of *command definitions* (`COMMANDS` registry: binary names, categories, per-command rules) from *context policies* (`CONTEXT_POLICIES`: which categories are allowed in each execution context). For example, the `local` context blocks file-reading commands (`cat`, `ls`, `find`) because the agent has dedicated file tools with path access control, while the `pod` context (remote container) allows these commands. This decoupling makes the ∼80-binary allowlist maintainable: adding a command requires specifying its category and per-command rules once; context policies determine where it may execute.

Layer 3: Three-Layer Output Sanitization Command outputs may contain sensitive data (JWT tokens, connection strings, PEM certificates) that would enter the LLM's context window if not intercepted. Our three-layer sanitization pipeline addresses this:

1. **Pre-execution analysis** (`analyzeOutput`): Before execution, examines the command and arguments to de-

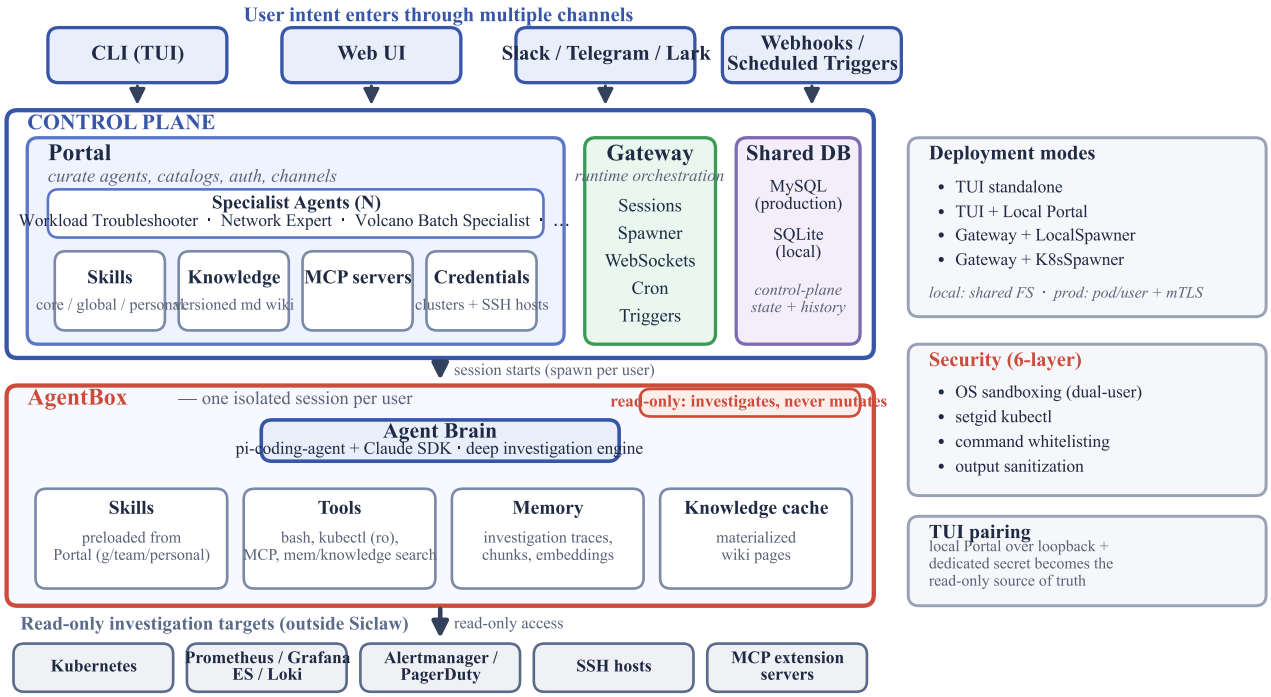

Figure 1: SICLAW system overview. User intent enters through four channels and reaches the *control plane* (Portal, Gateway, shared DB), which curates specialist agents, skills, knowledge, MCP servers, and credentials. Each user session spawns an isolated *AgentBox*—a Kubernetes pod in production, in-process locally, or embedded in a standalone TUI—containing the agent brain, preloaded skills, read-only tools, a private memory database, and a materialized knowledge cache. Every target the AgentBox touches (Kubernetes, observability stacks, SSH hosts, MCP servers) is accessed *read-only*: the agent investigates but never mutates. Four deployment modes trade isolation for convenience, and a six-layer security architecture (detailed in Figure 2) constrains all agent actions. Skills, knowledge, and credentials are provisioned per session from the control plane.

termine the appropriate sanitization strategy. Returns an `OutputAction` specifying which sanitizer function to apply.

2. **Post-execution redaction** (`applySanitizer`): Applies the pre-determined sanitizer to the command output. Sensitive patterns include environment variable names matching `*_API_KEY`, `*_SECRET`, `ANTHROPIC_*`, `DATABASE_URL`, and value patterns matching JWT tokens (`eyJ...`), PEM blocks (`-----BEGIN`), and connection strings (`postgres://`, `mongodb://`).
3. **Pipeline fallback**: For multi-command pipelines (*e.g.*, `kubectl get secret -o yaml | jq .data`), downstream commands (`jq`) may have no registered sanitization rule. The pipeline fallback applies line-level pattern-based redaction to the final output, catching sensitive data that Layers 1–2 missed.

Layers 4–6: Container Hardening, File I/O, Environment **Layer 4 (Container hardening):** The AgentBox pod drops all Linux capabilities except the five required for the dual-user model (`SETUID`, `SETGID`, `CHOWN`, `FOWNER`, `AUDIT_WRITE`), uses a read-only root filesystem, and disables the Kubernetes service account token mount.

Layer 5 (File I/O restrictions): The agent's file tools enforce `assertPathAllowed()` to scope reads/writes to designated directories, preventing the agent from using its own tools to access credential files.

Layer 6 (Environment sanitization): Before spawning child processes, `sanitizeEnv()` strips API keys, tokens, and sensitive environment variables from the process environment, preventing credential leakage via `/proc/{pid}/environ`.

The security architecture is validated by 184 unit tests covering the command validator (97 tests), output sanitizer (48 tests), security pipeline integration (15 tests), guard pipeline (16 tests), and tool call repair (8 tests), all passing.

Guard Pipeline for Agent Reliability

LLM agents exhibit systematic output unreliability that existing SRE systems do not address: malformed tool call JSON, missing tool call IDs, context window overflow from large command outputs, and corrupted streaming events. Every benchmark documents these issues (Chen et al. 2025b; Clark et al. 2026), yet no system provides a systematic solution.

The guard pipeline intercepts agent messages at four lifecycle stages:

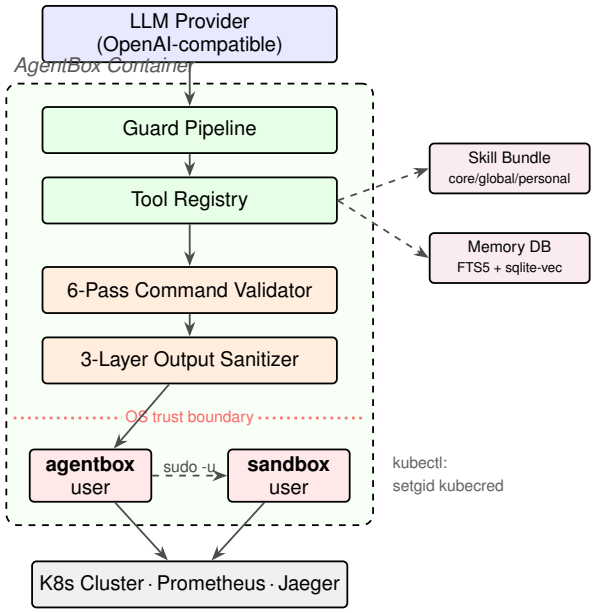

Figure 2: SICLAW architecture. LLM-generated commands flow through the guard pipeline, tool registry, 6-pass command validator, and 3-layer output sanitizer before crossing the OS trust boundary. Child processes run as the unprivileged `sandbox` user; `kubectl` reaches the kubeconfig via setgid. Skills and memory branch off the tool registry.

Input Guards. Transform the message history before each LLM API call. Guards include: (1) *sanitize-tool-calls*: drops tool call blocks missing required fields (`id`, `name`, or `input`); (2) *repair-tool-use-pairing*: ensures every `toolCall` has a matching `toolResult`, inserting synthetic error results where needed.

Context Guards. Enforce context budget with a three-tier strategy: (1) truncate any single tool result exceeding 50% of the context window; (2) compact oldest tool results when total context exceeds 75%; (3) raise a preemptive overflow signal at 90%, triggering compaction before the LLM API rejects the request.

Output Guards. Repair streaming response events in-flight: (1) *trim-tool-call-names*: trims whitespace and assigns fallback IDs to unnamed tool calls; (2) *repair-malformed-args*: extracts balanced JSON from partial `toolcall_delta` events using bracket counting.

Persist Guards. Validate messages before writing to session history: (1) sanitize tool calls on write; (2) track tool call/result pairing; (3) insert synthetic error results for orphaned calls; (4) truncate oversized results (>400KB).

The pipeline uses a *single-wrap-per-hook* architecture: each of the brain's three hook points (`streamFn`, `appendMessage`, `transformContext`) is wrapped exactly once with a pipeline that orchestrates all registered guards sequentially. This avoids the debugging complexity of nested monkey-patching while enabling fine-grained per-stage control.

Algorithm 1 Six-Pass Command Validation Pipeline

Require: Command string c, execution context $\xi \in \{\texttt{local}, \texttt{node}, \texttt{pod}\}$
Ensure: Validated command or rejection
1: **Pass 1 — Shell Operators:** Block `$()`, backticks, `<()`, `>()`, redirections, embedded newlines
2: **Pass 2 — Pipeline Extraction:** Parse c into pipeline stages $[s_1|s_2|\ldots]$ with position tracking
3: **for** each stage s_i in pipeline **do**
4: **Pass 3 — Binary Whitelist:** Verify binary$(s_i) \in \mathcal{W}_\xi$ where $\mathcal{W}_\xi$ is the context-specific allowlist
5: **Pass 4 — Subcommand Validation:** For `kubectl`, verify subcommand $\in$ {`get`, `describe`, `logs`, ...} (13 read-only commands)
6: **Pass 5 — Flag Restrictions:** Apply per-command rules: `pipeOnly`, `noFilePaths`, `blockedFlags`, `allowedFlags`
7: **end for**
8: **Pass 6 — Sensitive Paths:** Block commands targeting credential/configuration file paths
9: **return** approved command

Multi-Tenant Runtime Polymorphism

As shown in Figure 1, SICLAW supports four runtime modes sharing one agent core, trading isolation for convenience: (1) **standalone TUI**; (2) **TUI + local Portal** (read-only Portal snapshot over loopback with three-gate authentication); (3) **Gateway + LocalSpawner** for multi-user local development on a shared filesystem; and (4) **Gateway + K8sSpawner** for production, with per-user pod isolation and mTLS certificates encoding `userId`/`workspaceId`. Two invariants matter. On the shared filesystem (mode 3), `materialize()` must never be called—it would overwrite all users' skill directories. In production (mode 4), per-pod isolation is the multi-tenant boundary. All modes implement the `BoxSpawner` interface; the unified agent core operates identically across them.

Extensible Skill Architecture

Existing SRE agents use fixed tool sets (Chen et al. 2025b; Clark et al. 2026) or static SOPs (Pei et al. 2025). SICLAW instead organizes diagnostic skills into a four-tier hierarchy ordered by user proximity (personal > skillset > global > core): **core** skills are baked into the container image (versioned with code), **global** skills are organization-wide, **skillset** skills are team-scoped (development only, promoted to global for production), and **personal** skills are per-user. The skill bundle contract (`buildSkillBundle`) transmits only non-core skills, as core skills exist on disk.

Three-Gate Review. Because skills execute as code, new skills pass three independent safety gates before production: (1) *static analysis* against 22 danger patterns (shell injection, credential access, unsafe operations); (2) *AI semantic review* of skill intent; and (3) *human approval*. Unapproved skills cannot execute regardless of tier—enabling safe organizational knowledge accumulation, unlike the fixed tool sets of prior systems.

Hybrid Memory System

Each AgentBox maintains a private memory database (SQLite with FTS5 and sqlite-vec) holding structured investigation records with a full lifecycle (create → update → resolve → reference). Search combines vector similarity and keyword matching:

$$\text{score} = w_v \cdot \cos(\mathbf{q}, \mathbf{d}) + w_f \cdot \text{BM25}(q, d) + w_t \cdot \text{decay}(t) \quad (1)$$

with configurable weights (w_v=0.70, w_f=0.30) and temporal decay; results are reranked via Maximal Marginal Relevance. Knowledge documents are chunked on heading boundaries (∼400 tokens, ∼80 overlap), embedded, and dual-indexed—letting the agent recall prior investigations rather than starting each incident cold.

Evaluation

We evaluate SICLAW along four axes: (1) diagnostic capability on 100 Kubernetes fault scenarios, (2) security effectiveness via red-team testing with layer ablation, (3) security–capability tradeoff analysis, and (4) GPU/RDMA infrastructure fault diagnosis on 10 hardware-grounded scenarios.

Diagnostic Evaluation

Setup We construct a benchmark of 100 Kubernetes fault diagnosis cases across 10 categories (Table 3), inspired by the evaluation methodology of SREGym (Clark et al. 2026) and AIOpsLab (Chen et al. 2025b). Cases are deployed on a real Kubernetes cluster with 368 nodes (including 2 GPU nodes with H100 NVLink 80GB), Volcano scheduling APIs, and Calico NetworkPolicy. The agent operates under a *read-only constraint*: no cluster modifications are permitted.

Each case injects a specific fault and provides the agent with a symptom-level incident prompt (*e.g.*, "Pod `c031-pending` is stuck in Pending state"). The agent autonomously investigates using `kubectl`, `grep`, `jq`, and other whitelisted tools.

Scoring Following SREGym's checklist methodology, we evaluate across five dimensions:

- **Localization**: Correctly identified the target resource (pod, deployment, service, PVC, etc.)
- **Mechanism**: Explained the root fault cause, not just symptoms
- **Scope**: Specified impact (single resource, service path, compound multi-resource)
- **Evidence**: Cited concrete observables (events, YAML, logs, scheduler state)
- **Remediation**: Provided safe, root-cause-matched fix recommendation

Each dimension is scored 0–1 using an LLM-as-judge with oracle ground truth. A case passes if its average score exceeds 0.65.

Results SICLAW achieves 90.0% pass rate with average score 0.818 (Table 3). CrashLoop (100%, 0.938) and Config (100%, 0.895) show near-perfect diagnosis. Compound cases are hardest (71.4%, 0.703), requiring identification of all root causes. Network-DNS (70.0%) has the longest investigation time (95.6s, 8.4 tool calls). Tool efficiency varies widely: storage needs only 2.5 calls while Volcano-GPU requires 12.5 to trace the scheduling hierarchy.

Table 3: Diagnostic evaluation results across 10 fault categories. Categories span three difficulty levels: easy (single-fault), medium (multi-layer), and hard (compound/GPU-specific).

Category	n	Pass%	Score	Tools	Time
Image Pull	10	90.0	0.843	8.1	21.9s
CrashLoop	10	100.0	0.938	7.3	26.3s
Config	10	100.0	0.895	6.3	21.7s
Scheduling-GPU	15	80.0	0.808	7.7	48.0s
Storage	10	100.0	0.863	2.5	30.9s
Service-Readiness	12	100.0	0.758	6.0	62.0s
Network-DNS	10	70.0	0.780	8.4	95.6s
Controller	8	87.5	0.726	5.6	64.7s
Volcano-GPU	8	100.0	0.833	12.5	77.6s
Compound	7	71.4	0.703	11.6	73.3s
Overall	**100**	**90.0**	**0.818**	**7.8**	**56.1s**

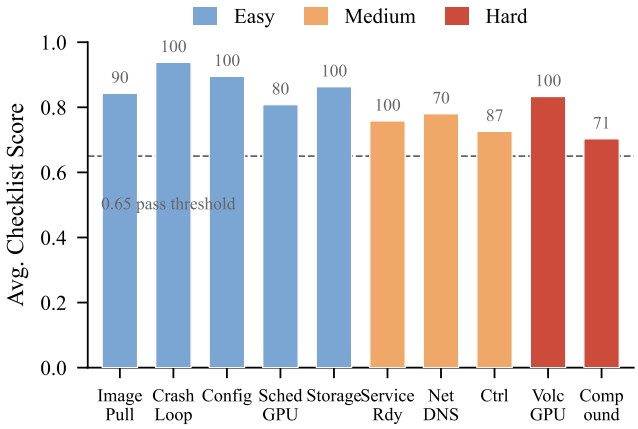

Figure 3: Diagnostic scores by category. Numbers above bars show pass rates. The dashed line marks the 0.65 pass threshold. Colors indicate difficulty: blue (easy), orange (medium), red (hard).

Security Red-Team Evaluation

We design 30 attack scenarios across the four threat categories (T1–T4) to evaluate the defense-in-depth architecture. Each attack is tested against the full security pipeline, and we attribute blocking to the specific layer that intercepts it.

The 30 scenarios span: **T1** credential reading (8: file access, env vars, `/proc` traversal, kubectl secret output); **T2** data exfiltration (8: curl POST, wget, reverse shells, command substitution, redirections); **T3** privilege escalation (8: kubectl write commands, sudo, blocked binaries); **T4** output leakage (6: secret/configmap/pod data, JWT tokens, PEM keys, connection strings).

Table 4: Security red-team results. All 30 attacks are blocked. "Primary Layer" indicates the first layer that intercepts each attack.

Threat category	Primary blocking layer(s)	Blocked
T1: Credential read	L2 (6), L4 (1), L6 (1)	8/8
T2: Exfiltration	L2 (3), L1 (3), L5 (2)	8/8
T3: Priv. escalation	L5 (5), L2 (3)	8/8
T4: Output leakage	L4 (6)	6/6
Overall	all layers	**30/30**

Table 5: Per-layer independent coverage. Each row shows attacks blocked if only that layer existed. No single layer achieves 100%; the combined architecture does.

Security Layer	Blocked	Coverage
L1: Shell Operator Validation	3/30	10.0%
L2: Binary Whitelist	21/30	70.0%
L3: Container Hardening (OS)	—	(OS-level)
L4: Output Sanitization	4/30	13.3%
L5: Command Restrictions	14/30	46.7%
L6: Sensitive Path Patterns	7/30	23.3%
Combined (all layers)	**30/30**	**100.0%**

Results **All 30 attack scenarios are blocked** (100% interception rate). Table 4 shows the per-category results and primary blocking layer.

Security Layer Ablation

To validate the defense-in-depth design, we test each security layer independently: "If *only* this layer existed, how many of the 30 attacks would it block?" Table 5 shows the results.

Key findings: (1) No single layer exceeds 70% coverage; L2 (whitelist) misses all kubectl write attacks (caught by L5) and output leakage (caught by L4). (2) 18/30 attacks (60%) are caught by 2+ layers, demonstrating true defense-in-depth. (3) 12 attacks are caught by exactly one layer — removing it would create a vulnerability (*e.g.*, kubectl write commands are caught *only* by L5's subcommand check).

Diagnostic Performance Analysis Table 6 shows the per-dimension score breakdown across categories, revealing systematic patterns in agent diagnostic behavior.

Key patterns: Remediation is perfect (1.000) across all categories. Scope is the weakest dimension (0.481), especially for service-readiness (0.133) and controller (0.200) where multi-resource impact chains are hard to articulate. 9/10 failures are localization format issues (the agent explains the mechanism correctly but omits the exact resource name), not diagnostic capability gaps. Difficulty scaling is non-linear: easy 92.7%, medium 83.3%, hard 90.5% — the hard-case improvement with Claude Sonnet 4.6 confirms model capability as the primary factor.

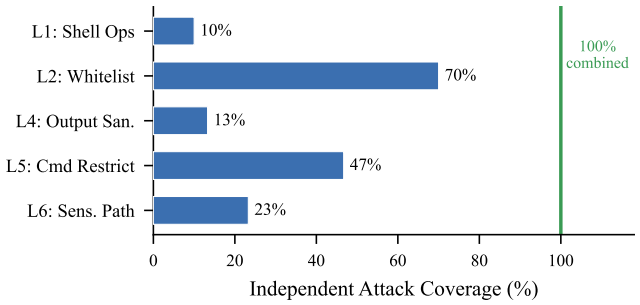

Figure 4: Independent coverage of each security layer. No single layer exceeds 70%. The green line marks the combined 100% coverage.

Security–Capability Tradeoff Analysis

A critical question is: *do security constraints degrade diagnostic capability?* We analyze 739 tool calls across 100 cases (Table 7). The agent used only 6 of 13 allowed kubectl subcommands; 4 `kubectl exec` attempts were blocked. We compiled 16 representative unsafe commands LLMs naturally attempt (Clark et al. 2026); **15/16 (93.8%) are blocked**. Extrapolating, an unconstrained agent would generate ∼**242 security violations across 100 cases** (2.4/case). The output sanitizer triggered **48 times**, redacting sensitive data before it entered the LLM context.

Experiment B: Accuracy Cost of Security We examine all 10 failed cases to determine whether any failure was caused by a security constraint blocking a diagnostically useful command. For each failure, we check: (1) whether any tool calls were rejected by the security pipeline, and (2) whether the failure dimensions (localization, mechanism) correspond to information that a blocked command would have provided.

Zero of 10 failures were caused or contributed to by security constraints. The failure root causes are:

- **Localization format** (9 cases): The agent correctly identified the root cause mechanism (average mechanism score 0.82) but omitted the exact resource name from its final answer — a format compliance issue, not a diagnostic capability gap.
- **Mechanism understanding** (1 case, c082): The agent localized the correct resource but failed to explain the controller-level root cause.

This result demonstrates that the security architecture's whitelisted commands (`get`, `describe`, `logs`, `events`, `top`) are *sufficient* for Kubernetes fault diagnosis. The 13 allowed kubectl subcommands cover the standard read-only diagnostic workflow; the excluded write commands (`apply`, `delete`, `patch`, `exec`) are not needed for diagnosis.

GPU/RDMA Infrastructure Fault Diagnosis

A critical gap in existing SRE agent benchmarks is the absence of GPU and RDMA network fault scenarios (Clark et al. 2026; Chen et al. 2025b). No published benchmark

Table 6: Average scores per category and scoring dimension. Bold indicates scores below 0.70. Remediation achieves perfect scores across all categories. Scope is the weakest dimension overall (0.481).

Category	n	**Loc.**	**Mech.**	**Scope**	**Evid.**	**Remed.**	**Total**
Image Pull	10	0.900	0.867	**0.440**	0.982	1.000	0.843
CrashLoop	10	1.000	0.869	0.960	0.890	1.000	0.938
Config	10	1.000	0.964	**0.440**	0.930	1.000	0.895
Scheduling-GPU	15	0.800	0.832	**0.547**	0.907	1.000	0.808
Storage	10	1.000	0.797	**0.567**	0.927	1.000	0.863
Service-Readiness	12	1.000	0.844	**0.133**	**0.567**	1.000	0.758
Network-DNS	10	0.700	0.772	0.700	0.890	1.000	0.780
Controller	8	1.000	**0.679**	**0.200**	**0.619**	1.000	0.726
Volcano-GPU	8	1.000	0.817	**0.458**	0.792	1.000	0.833
Compound	7	**0.571**	0.882	**0.286**	0.829	1.000	0.703
Overall	**100**	0.900	0.834	**0.481**	0.837	1.000	**0.818**

Table 7: Security–capability tradeoff. The security pipeline prevents an estimated 242 violations per 100 cases while causing zero diagnostic failures.

Metric	**Value**
Total tool calls (100 cases)	739
Bash / kubectl calls	504 / 501
Unique kubectl subcommands used	6 of 13 allowed
Simulated unsafe commands blocked	15/16 (93.8%)
Estimated violations per 100 cases	∼242
Output sanitization events	48
Diagnostic pass rate (with security)	90.0%
Failures caused by security constraints	**0/10**
Accuracy cost of security	**0%**

evaluates agent diagnosis of GPU hardware errors (Xid events, ECC failures, NVLink degradation) or RDMA fabric issues (link flapping, PFC storms, QP errors) — despite these being the dominant failure modes in production GPU clusters.

Methodology We design 10 fault scenarios based on observable-signal simulation: rather than injecting hardware faults (which requires physical access), we reproduce the *diagnostic signals* an SRE agent would observe — kernel logs (dmesg), GPU telemetry (nvidia-smi), RDMA counters (ibstat, ethtool), and NCCL error logs — as Kubernetes ConfigMaps and Pod annotations in the evaluation namespace. This approach is consistent with the agent's diagnostic interface: it reads `kubectl` output, not hardware registers.

Scenarios are grounded in production fault data from: Xid error characterization on H100 GPUs (11.7M GPU-hours, Cui et al., 2025), RDMA datacenter congestion patterns (Ghorbani et al., IMC 2025), RDMA failover challenges (SHIFT, 2025), and production GPU cluster reliability at ByteDance (ByteRobust, SOSP 2025).

Results SICLAW achieves **100% pass rate** (10/10, avg. 0.923) on GPU/RDMA scenarios (Table 8). GPU hardware faults (avg. 0.920) are diagnosed accurately: the agent maps Xid codes to specific GPUs, reads NVLink topology, and distinguishes thermal throttling from genuine OOM. RDMA faults (avg. 0.938) are correctly attributed to link-layer issues via ibstat/PFC counters. In compound case g09, the agent identifies *both* independent root causes (GPU ECC + RDMA congestion). To our knowledge, this is the **first evaluation of an SRE agent on GPU/RDMA infrastructure faults** (Yu et al. 2024a; Cui et al. 2025).

Table 8: GPU/RDMA fault diagnosis (10 cases, Claude Sonnet 4.6). Observable-signal simulation of hardware faults grounded in production data from Cui et al. (2025), Ghorbani et al. (IMC'25), and SHIFT (2025).

ID	**Fault Type**	**Score**	**Time**
GPU Hardware (5 cases)			
g01	Xid 48 double-bit ECC error	0.961	30s
g02	NVLink degradation (40% slowdown)	0.991	29s
g03	Thermal throttling → CUDA OOM	0.926	37s
g06	ECC row remapping exhausted	0.921	32s
g07	Silent GPU clock degradation	0.799	36s
RDMA Network (3 cases)			
g04	InfiniBand link flap → NCCL timeout	0.972	28s
g05	PFC storm (cluster-wide congestion)	0.895	30s
g08	RDMA QP error state (training hang)	0.945	65s
Compound GPU+RDMA (2 cases)			
g09	GPU ECC + RDMA congestion	0.975	54s
g10	Volcano gang sched. + node drain	0.841	238s
Overall	**10/10 passed (100%)**	**0.923**	**58s**

Comparison. On SREGym, Claude Code achieves 60.7% and Stratus 54.8% E2E success (Clark et al. 2026). SICLAW achieves 90% on 100 K8s cases and 100% on 10 GPU/RDMA cases. The key differentiator is safety: **neither provides any documented security model** for shell access, while an unconstrained agent generates ∼2.4 security violations per case.

Discussion

Agent-as-Untrusted-Code. Existing SRE agent research treats the LLM as trusted. Our production experience shows

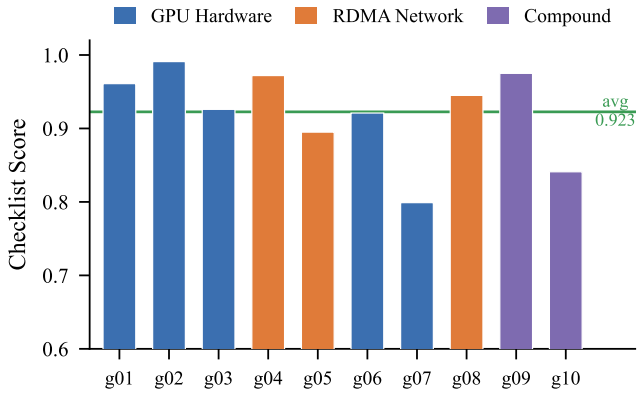

Figure 5: GPU/RDMA fault diagnosis scores. All 10 cases pass (above 0.65 threshold). Compound cases (g09, g10) combining GPU and RDMA faults are the most challenging.

this is untenable: even under explicit read-only instructions, the agent attempted 4 `kubectl exec` commands via normal hallucination. An unconstrained agent would generate ∼2.4 security violations per case (∼242 total), yet our defense-in-depth catches all of them — with **zero accuracy cost** (0/10 failures caused by security constraints). The agent needed only 6 of 13 allowed kubectl subcommands; all failures were formatting or reasoning issues orthogonal to security.

GPU/RDMA Fault Diagnosis. Production GPU clusters experience 5K–60K RDMA link flaps daily, and H100 GPUs show 3.2× worse uncorrectable memory error rates than A100 (Cui et al. 2025), yet no benchmark covers these faults (Yu et al. 2024a). Our observable-signal simulation — injecting diagnostic signals (dmesg, nvidia-smi, ibstat, NCCL traces) as ConfigMaps — enables evaluation without specialized hardware. The 100% pass rate suggests LLM agents can effectively diagnose GPU/RDMA faults when given appropriate telemetry.

Generalizability. The security architecture is not Kubernetes-specific. The dual-user model, command validation pipeline, and output sanitization apply to any LLM agent executing shell commands. The context-policy decoupling allows supporting new execution contexts (*e.g.*, cloud CLIs, database tools) by defining policies without modifying command definitions.

Limitations. The evaluation uses a single cluster topology; cross-cluster environments remain future work. The red-team covers 30 scenarios but not multi-turn prompt injection chains. The skill review gate requires human involvement. The memory system is tuned for personal scale (∼1000 chunks). The comparison with Stratus and Claude Code relies on published results rather than direct reproduction.

Conclusion

We presented SICLAW, the first production-grade SRE agent system that treats the LLM as untrusted code. Its six-layer defense-in-depth architecture—OS-level dual-user isolation with setgid transitive privilege, six-pass command validation, three-layer output sanitization, and a four-stage guard pipeline—blocked 100% of 30 red-team attacks at **zero accuracy cost** (no diagnostic failure was caused by a security constraint). Layer ablation confirms no single layer exceeds 70% coverage, and the output sanitizer intercepted 48 sensitive exposures across 100 runs.

SICLAW supports three runtime modes sharing a unified agent core, with a four-tier skill architecture for organizational knowledge. It achieves 90% diagnostic pass rate (avg. 0.818) on 100 Kubernetes faults and 100% (avg. 0.923) on 10 GPU/RDMA infrastructure faults — the first such evaluation for SRE agents. Because even non-adversarial agents generate ∼2.4 security violations per case through normal hallucination, we argue the agent-as-untrusted-code paradigm is essential for production deployment, and hope SICLAW provides both a foundation and a research agenda for secure autonomous operations.

References

Chen, Y.; Pan, J.; Clark, J.; Su, Y.; Zheutlin, N.; Bhavya, B.; Arora, R.; Deng, Y.; Jha, S.; and Xu, T. 2025a. Stratus: A Multi-agent System for Autonomous Reliability Engineering of Modern Clouds. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Chen, Y.; Shetty, M.; Somashekar, G.; Ma, M.; Simmhan, Y.; Mace, J.; Bansal, C.; Wang, R.; and Rajmohan, S. 2025b. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *Proceedings of Machine Learning and Systems (MLSys)*.

Chen, Y.; Xie, H.; Ma, M.; Kang, Y.; Gao, X.; Shi, L.; Cao, Y.; Gao, X.; Fan, H.; Wen, M.; et al. 2024. RCACopilot: Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.

Clark, J.; Su, Y.; Pial, S. M. R.; Tian, Y.; Gniedziejko, L.; Jacobsen, H.-A.; Chen, Y.; and Xu, T. 2026. SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios. *arXiv preprint arXiv:2506.XXXXX*.

Cui, R.; et al. 2025. Characterizing GPU Resilience and Impact on AI/HPC Systems. *arXiv preprint arXiv:2503.11901*.

Gan, Y.; Zhang, Y.; Cheng, D.; Shetty, A.; Rathi, P.; Katarki, N.; Bruno, A.; Hu, J.; Ritchken, B.; Jackson, B.; et al. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

Jha, S.; Arora, R. R.; Watanabe, Y.; Yanagawa, T.; Chen, Y.; Clark, J.; et al. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *Proceedings of the International Conference on Machine Learning (ICML)*.

Jiang, Y.; Zhang, C.; He, S.; Yang, Z.; Ma, M.; Qin, S.; Kang, Y.; Dang, Y.; Rajmohan, S.; Lin, Q.; and Zhang, D. 2024. Xpert: Empowering Incident Management with Query Recommendations via LLMs. In *Proceedings of the International Conference on Software Engineering (ICSE)*.

Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*.

Pei, C.; Wang, Z.; Liu, F.; Li, Z.; Liu, Y.; He, X.; Kang, R.; Zhang, T.; Chen, J.; Li, J.; Xie, G.; and Pei, D. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *Companion Proceedings of the Web Conference (WWW)*.

Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *International Conference on Learning Representations (ICLR)*.

Wan, M.; et al. 2025. ByteRobust: Robust LLM Training Infrastructure. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.

Wang, Y.; Yu, G.; Huang, H.; Wang, Z.; Huang, Y.; Chen, P.; and Lyu, M. R. 2026. Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems. *arXiv preprint arXiv:2602.XXXXX*.

Wang, Z.; Liu, Z.; Zhang, Y.; Zhong, A.; Fan, J.; Wu, L.; and Wen, Q. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented LLMs. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*.

Xu, J.; Zhang, Q.; Zhong, Z.; He, S.; Zhang, C.; Lin, Q.; Pei, D.; He, P.; Zhang, D.; and Zhang, Q. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures? In *International Conference on Learning Representations (ICLR)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.

Yu, G.; et al. 2024a. A Survey on Failure Analysis and Fault Injection in AI Systems. *ACM Transactions on Software Engineering and Methodology*.

Yu, Z.; Ma, M.; Zhang, C.; Qin, S.; Kang, Y.; Bansal, C.; Rajmohan, S.; Dang, Y.; Pei, C.; Pei, D.; Lin, Q.; and Zhang, D. 2024b. MonitorAssistant: Simplifying Cloud Service Monitoring via Large Language Models. In *Companion Proceedings of the International Symposium on Foundations of Software Engineering (FSE)*.

Zhang, X.; Mittal, T.; Bansal, C.; Wang, R.; Ma, M.; Ren, Z.; Huang, H.; and Rajmohan, S. 2024. FLASH: A Workflow Automation Agent for Diagnosing Recurring Incidents. *arXiv preprint arXiv:2410.XXXXX*.

Zhang, X.; Wang, Q.; Li, M.; Yuan, Y.; Xiao, M.; Zhuang, F.; and Yu, D. 2025. TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent With Multi-Modality Observation Data. *IEEE Transactions on Services Computing*, 18(6).

Zhou, X.; et al. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proceedings of the VLDB Endowment*.